

软件过程改进方案设计文档

一、引言

随着生成式人工智能（Generative AI）、大语言模型（Large Language Model, LLM）以及智能软件工程工具的快速发展，AI技术正逐步渗透到软件开发生命周期的各个阶段。对于本项目——“网页购物系统”，原软件过程采用简化的瀑布+迭代混合模型，虽然该过程具有轻量化、易实施等特点，但仍存在需求分析效率低、设计经验不足、编码工作量大、测试资源紧张等问题。

需求分析（REQ-01）、系统设计（DES-01）、编码实现（IMP-01）和测试验证（TST-01）四个环节具有较高的AI应用价值。其中，编码实现与测试验证被确定为重点改进对象。

本方案旨在将AI技术融入现有软件过程，在保证过程可执行性的基础上，提高开发效率、降低人为错误率，并增强软件质量。

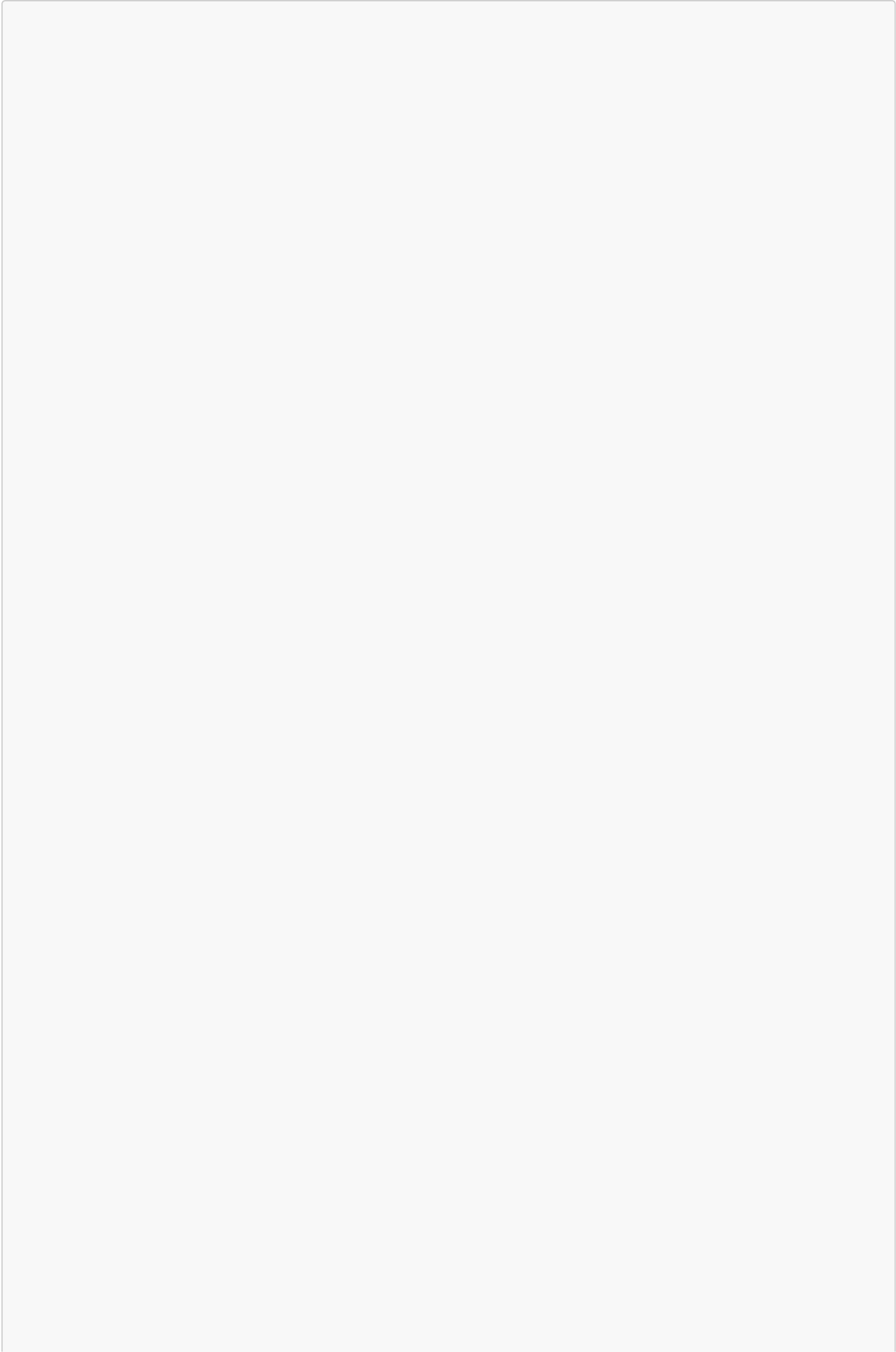
二、改进目标

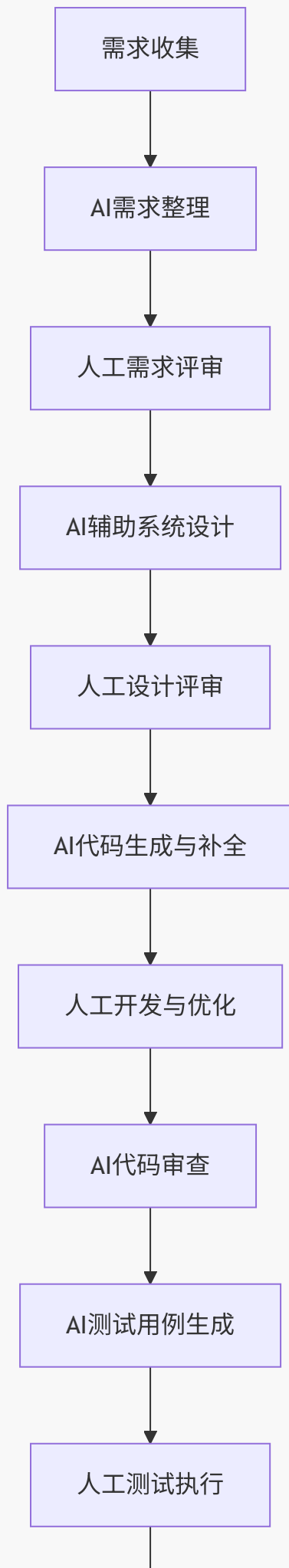
本次过程改进主要实现以下目标：

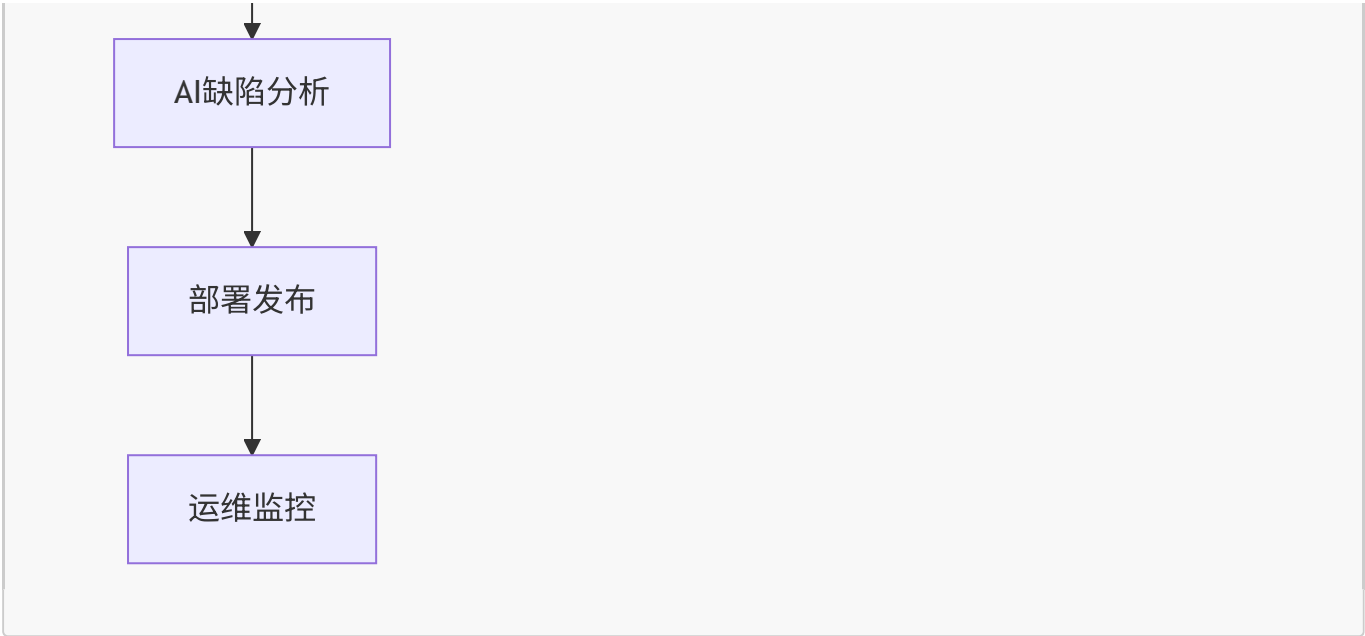
- 提高需求分析效率，降低需求遗漏率；
- 提升系统设计质量，减少设计缺陷；
- 提高编码效率和代码质量；
- 降低测试工作量，提高测试覆盖率；
- 建立适合小型团队的人机协同开发模式；
- 在不增加过多管理成本的前提下实现过程智能化升级。

三、AI驱动的软件过程重构

3.1 改进后总体流程







四、关键改进环节设计

4.1 需求分析过程改进（REQ-AI）

原过程

- 小组讨论需求
- 编写SRS
- 绘制用例图

改进方案

引入ChatGPT辅助需求分析。

主要活动：

1. 收集需求讨论记录；
2. AI自动整理功能需求；
3. AI补充遗漏需求；
4. AI生成用户故事；
5. AI生成Mermaid用例图；
6. 小组评审确认。

输出物

- AI生成需求清单
- 用户故事集
- 用例图
- SRS文档

预期效果

指标	改进前	改进后
SRS编写时间	3-4天	1-2天
用例图绘制	1天	1小时
需求遗漏率	基准	降低30%

4.2 系统设计过程改进（DES-AI）

原过程

- 数据库设计
- API设计
- 架构设计

改进方案

使用ChatGPT辅助设计。

主要活动：

1. 输入功能需求；
2. AI生成ER图设计方案；
3. AI生成数据库SQL；
4. AI生成RESTful API清单；
5. AI生成系统架构图；
6. 开发人员评审修订。

输出物

- 数据库设计文档
- ER图
- API接口文档
- 系统架构图

预期效果

指标	改进前	改进后
数据库设计时间	3天	1天
API设计时间	2天	0.5天
设计缺陷率	基准	降低25%

4.3 编码实现过程改进（IMP-AI）

原过程

- 手工编写代码

- 人工Code Review

改进方案

引入GitHub Copilot/ChatGPT/DeepSeek。

主要活动：

1. AI生成CRUD代码；
2. AI生成前端组件；
3. AI生成单元测试；
4. AI辅助重构；
5. AI代码审查。

输出物

- 源代码
- 自动生成测试代码
- AI审查报告

预期效果

指标	改进前	改进后
编码效率	基准	提升40%-60%
CRUD开发时间	3周	1周
单元测试覆盖率	75%	85%以上

4.4 测试验证过程改进（TST-AI）

原过程

- 人工设计测试用例
- 人工回归测试

改进方案

使用ChatGPT/DeepSeek生成测试用例。

主要活动：

1. 输入需求说明；
2. AI生成测试用例；
3. AI补充边界场景；
4. 人工执行测试；
5. AI辅助分析测试结果；
6. 自动生成测试报告。

输出物

- 测试用例表
- 测试报告
- 缺陷分析报告

预期效果

指标	改进前	改进后
测试用例编写时间	3天	1天
测试覆盖率	75%	90%以上
回归测试时间	2天	0.5天

五、新增角色设计

AI过程协调员

职责：

- 负责AI工具使用规范；
- 审核AI输出结果；
- 维护提示词模板。

AI测试分析师

职责：

- 利用AI生成测试资产；
- 分析测试结果；
- 编写验证报告。

六、工具链选型

环节	工具	核心功能
需求分析	ChatGPT	需求整理、用户故事生成
系统设计	ChatGPT、DeepSeek	ER图与API设计
编码实现	GitHub Copilot	代码生成与补全
代码审查	ChatGPT	Bug分析与优化建议
测试验证	ChatGPT	测试用例生成
项目管理	GitHub、Notion	过程协同

七、风险评估与应对策略

7.1 数据安全风险

风险：

将源码或需求上传至云端大模型可能导致信息泄露。

措施：

- 不上传敏感数据；
 - 使用脱敏处理；
 - 优先使用可信平台。
-

7.2 模型幻觉风险

风险：

AI生成错误设计或测试方案。

措施：

- 所有输出必须人工审核；
 - 关键设计需团队评审。
-

7.3 技术依赖风险

风险：

开发人员过度依赖AI。

措施：

- 保留人工设计与审核环节；
 - 定期进行独立开发训练。
-

7.4 输出质量不稳定风险

风险：

不同Prompt可能产生不同结果。

措施：

- 建立统一Prompt模板；
 - 制定AI输出质量标准。
-

八、总结

本方案针对网页购物系统项目的需求分析、系统设计、编码实现和测试验证四个关键环节进行了AI化改造设计。通过引入ChatGPT、GitHub Copilot、DeepSeek等工具，实现了从需求到测试的全流程智能辅助开发。

预计整体开发效率能够提升40%左右，测试用例编写时间减少60%左右，需求遗漏率 and 设计缺陷率明显下降。该方案符合小组的项目规模与开发周期要求，具有良好的可落地性和实践价值。